

REPASO

Fundamentos de SQL

SELECT · WHERE · GROUP BY · JOINS · Subqueries · CTEs · Esquema Estrella

Clase de repaso — aprox. 1 hora

Agenda — Repaso completo de SQL

	Datos de práctica	Tablas clientes y pagos (inspirado en Sakila)	5 min
Bloque 1	SELECT, WHERE, ORDER BY	Lo esencial más lo PRO: *, alias, DISTINCT, LIMIT	8 min
	Pregunta de negocio #1	Desafío intermedio con los datos de practica	5 min
Bloque 2	GROUP BY y Agregaciones	COUNT, SUM, AVG, MIN, MAX + HAVING	10 min
Bloque 3	JOINS	INNER, LEFT, RIGHT, FULL — diferencias con nuestras tablas	12 min
	Pregunta de negocio #2	Desafío con JOINS	5 min
Bloque 4	Subqueries y CTEs	Dividir queries complejas — WITH y subconsultas	8 min
Bloque 5	Funciones esenciales	CASE WHEN, fechas y texto	5 min
Bloque 6	Esquema Estrella	Repaso visual + pregunta de negocio + ingenieria inversa	8 min

Tablas de trabajo para toda la sesion

Datos de práctica

Tablas de practica — clientes y pagos

clientes: 7 filas (2 sin pagos) | pagos: 9 filas (2 huérfanos sin cliente)

clientes — tabla de clientes

```
CREATE TABLE clientes (  
  id_cliente INT PRIMARY KEY,  
  nombre VARCHAR(60),  
  email VARCHAR(80),  
  ciudad VARCHAR(40),  
  segmento VARCHAR(20)  
);  
  
INSERT INTO clientes VALUES  
(1,'Ana Garcia', 'ana@mail.com', 'Bogota', 'VIP'),  
(2,'Juan Lopez', 'juan@mail.com', 'Medellin','Frecuente'),  
(3,'Maria Ruiz', 'maria@mail.com', 'Cali', 'Frecuente'),  
(4,'Carlos Perez', 'carlos@mail.com', 'Bogota', 'Nuevo'),  
(5,'Laura Torres', 'laura@mail.com', 'Bogota', 'VIP'),  
(6,'Sofia Mora', 'sofia@mail.com', 'Cali', 'Nuevo'),  
(7,'Diego Rios', 'diego@mail.com', 'Medellin','Frecuente');
```

pagos — tabla de pagos

```
CREATE TABLE pagos (  
  id_pago INT PRIMARY KEY,  
  id_cliente INT,  
  fecha_pago DATE,  
  monto DECIMAL(10,2),  
  metodo VARCHAR(20)  
);  
  
INSERT INTO pagos VALUES  
(101,1,'2025-01-10',150000,'Credito'),  
(102,2,'2025-01-15', 80000,'Debito'),  
(103,3,'2025-02-01',120000,'Credito'),  
(104,4,'2025-02-10', 45000,'Efectivo'),  
(105,5,'2025-03-05',200000,'Credito'),  
(106,1,'2025-03-20',175000,'Credito'),  
(107,3,'2025-04-01', 90000,'Debito'),  
(108,10,'2025-04-15',60000,'Efectivo'),  
(109,11,'2025-05-01',30000,'Debito');
```

Clave para los JOINS: clientes 6 y 7 NO tienen pagos. Pagos 108 y 109 tienen id_cliente 10 y 11 que NO existen en clientes.

Lo esencial y lo PRO

Bloque 1 · SELECT, WHERE, ORDER BY

SELECT — anatomia completa y trucos PRO

No es solo 'traer columnas': hay detalles que hacen la diferencia

Orden de ejecucion en SQL:

1. FROM — de donde vienen los datos
2. WHERE — filtrar filas
3. GROUP BY — agrupar
4. HAVING — filtrar grupos
5. SELECT — que columnas mostrar
6. ORDER BY — ordenar el resultado
7. LIMIT — cuantas filas devolver

SELECT * no es malo... pero:

- trae todas las columnas aunque no las necesites
- en tablas grandes impacta el rendimiento
- el * se expande al ejecutar, no al escribir
- PRO: SELECT * al explorar, columnas al produccion

Ejemplos con nuestras tablas:

```
-- Alias: renombra columnas y tablas
SELECT nombre AS cliente,
       ciudad AS ubicacion
FROM clientes AS c;

-- DISTINCT: elimina duplicados
SELECT DISTINCT ciudad
FROM clientes;

-- ORDER BY + LIMIT: los 3 pagos mas altos
SELECT id_pago, monto, metodo
FROM pagos
ORDER BY monto DESC
LIMIT 3;

-- WHERE con multiples condiciones
SELECT * FROM clientes
WHERE ciudad = 'Bogota'
AND segmento = 'VIP';
```

PRO: WHERE se ejecuta ANTES que SELECT — no puedes filtrar por un alias definido en SELECT en la misma query.

PREGUNTA DE NEGOCIO

Pregunta #1 — Solo con SELECT y WHERE

¿Cuántos clientes distintos de Bogota
tienen segmento 'VIP' o 'Frecuente'?
Listalos por nombre en orden alfabetico.

SOLUCION

Solucion — Pregunta #1

```
SELECT nombre, ciudad, segmento  
FROM clientes  
WHERE ciudad = 'Bogota'  
      AND segmento IN ('VIP', 'Frecuente')  
ORDER BY nombre ASC;
```


Resumir datos: COUNT, SUM, AVG, MIN, MAX, HAVING

Bloque 2 · GROUP BY y Agregaciones

GROUP BY — agrupar y resumir

La diferencia entre ver filas individuales y ver totales por categoria

Las 5 funciones clave:

COUNT(*) — cuantas filas hay
COUNT(col) — filas NO nulas
SUM(col) — suma total
AVG(col) — promedio
MIN / MAX — menor / mayor valor

HAVING vs WHERE:

WHERE filtra filas ANTES de agrupar
HAVING filtra grupos DESPUES de agrupar

→ No puedes usar SUM() en WHERE
→ Si puedes usar SUM() en HAVING

Regla de GROUP BY:

En SELECT solo pueden ir:

1. Las columnas del GROUP BY
2. Funciones de agregacion

Cualquier otra columna = ERROR

```
-- Total pagado y numero de pagos por metodo
SELECT metodo,
       COUNT(*)      AS num_pagos,
       SUM(monto)     AS total_pagado,
       AVG(monto)     AS promedio_pago
FROM pagos
GROUP BY metodo
ORDER BY total_pagado DESC;
```

```
-- Clientes con mas de un pago
-- HAVING filtra los grupos resultado
SELECT id_cliente,
       COUNT(*)      AS num_pagos,
       SUM(monto)    AS total_pagado
FROM pagos
GROUP BY id_cliente
HAVING COUNT(*) > 1
ORDER BY total_pagado DESC;
```

HAVING es la 'clausula olvidada': actua sobre grupos, no sobre filas. Se ejecuta despues del GROUP BY.

Combinar tablas — INNER, LEFT, RIGHT, FULL OUTER

Bloque 3 · JOINS

Los 4 tipos de JOIN — con nuestros datos

clientes (7 filas) + pagos (9 filas): resultados distintos segun el JOIN

INNER JOIN — solo los que coinciden en ambas tablas

Cientes 1–5 tienen pagos → aparecen

Cientes 6 (Sofia) y 7 (Diego) NO → no aparecen

Pagos huérfanos 108 y 109 → no aparecen

Resultado: solo las filas que tienen pareja en AMBAS tablas

LEFT JOIN — TODOS los de la tabla izquierda

Todos los clientes (1-7) aparecen siempre

Sofia y Diego aparecen con NULL en columnas de payments

Pagos huérfanos 108, 109 → NO aparecen

El más usado en análisis: 'dame todos los clientes, con o sin pagos'

RIGHT JOIN — TODOS los de la tabla derecha

Todos los pagos (1-9) aparecen siempre

Pagos 108 y 109 aparecen con NULL en columnas de customers

Poco usado: se puede lograr igual con LEFT cambiando el orden

FULL OUTER JOIN — absolutamente todos

Las 7 filas de customers + las 9 de payments

NULLs donde no hay coincidencia en cualquier lado

Útil para: detectar datos huérfanos, auditar integridad

En la práctica: 80% INNER JOIN, 19% LEFT JOIN. RIGHT y FULL casi nunca — pero los preguntan en evaluaciones.

JOINS en codigo

La columna ON es la llave: id_cliente une las dos tablas

```
-- INNER JOIN: clientes CON pagos
SELECT c.nombre,
       c.segmento,
       p.monto,
       p.metodo
FROM clientes c
INNER JOIN pagos p
      ON c.id_cliente = p.id_cliente
ORDER BY p.monto DESC;
-- Resultado: 7 filas (clientes 1,3 tienen 2 pagos)
-- Sofia y Diego NO aparecen
```

```
-- LEFT JOIN: TODOS los clientes
SELECT c.nombre,
       c.segmento,
       p.monto,
       p.metodo
FROM clientes c
LEFT JOIN pagos p
      ON c.id_cliente = p.id_cliente
ORDER BY c.nombre;
-- Resultado: 9 filas
-- Sofia y Diego aparecen con NULL en monto/metodo
```

```
-- Caso practico: clientes SIN ningun pago
-- Patron clasico: LEFT JOIN + WHERE IS NULL
SELECT c.nombre, c.ciudad
FROM clientes c
LEFT JOIN pagos p
      ON c.id_cliente = p.id_cliente
WHERE p.id_pago IS NULL;
```

```
-- JOIN con agregacion: total por cliente
SELECT c.nombre,
       COUNT(p.id_pago) AS num_pagos,
       SUM(p.monto)     AS total_pagado
FROM clientes c
LEFT JOIN pagos p
      ON c.id_cliente = p.id_cliente
GROUP BY c.nombre
ORDER BY total_pagado DESC NULLS LAST;
```

PRO: LEFT JOIN + WHERE col_derecha IS NULL = encontrar registros sin pareja. Patron muy usado en auditorias.

PREGUNTA DE NEGOCIO

Pregunta #2 — JOIN + GROUP BY + HAVING

¿Que clientes VIP o Frecuentes han pagado
mas de \$100.000 en total?
Muestra nombre, segmento y total pagado.

SOLUCION

Solucion — Pregunta #2

```
SELECT c.nombre, c.segmento,  
       SUM(p.monto) AS total_pagado  
FROM clientes c  
LEFT JOIN pagos p  
      ON c.id_cliente = p.id_cliente  
WHERE c.segmento IN ('VIP', 'Frecuente')  
GROUP BY c.nombre, c.segmento  
HAVING SUM(p.monto) > 100000  
ORDER BY total_pagado DESC;
```

Dividir queries complejas en pasos legibles

Bloque 4 · Subqueries y CTEs

Subquery vs CTE — dos formas de dividir una query

Ambas logran lo mismo; la diferencia es legibilidad y reusabilidad

Subquery — query dentro de otra query

En WHERE: filtra usando resultado de otra query

En FROM: usa el resultado como una tabla temporal

```
-- Subquery en WHERE
-- Clientes que han pagado mas que el promedio
SELECT nombre, ciudad
FROM clientes
WHERE id_cliente IN (
    SELECT id_cliente
    FROM pagos
    GROUP BY id_cliente
    HAVING SUM(monto) > (
        SELECT AVG(monto) FROM pagos
    )
);
```

CTE (WITH) — tabla temporal con nombre

Se define antes de la query principal con WITH

Mas legible, puede reutilizarse varias veces en la misma query

```
-- CTE: misma logica, mas clara
WITH clientes_top AS (
    SELECT id_cliente,
           SUM(monto) AS total
    FROM pagos
    GROUP BY id_cliente
    HAVING SUM(monto) > (
        SELECT AVG(monto) FROM pagos
    )
)
SELECT c.nombre, c.ciudad, ct.total
FROM clientes c
JOIN clientes_top ct
  ON c.id_cliente = ct.id_cliente;
```

Subquery vs CTE — cuando usar cada uno:

Subquery: query rapida, logica simple, solo se usa una vez en la query | CTE: logica compleja, se necesita reusar, trabajo en equipo (mas legible)

PRO: los CTEs se procesan de arriba hacia abajo. Puedes encadenar varios WITH separados por coma.

CASE WHEN · Fechas · Texto

Bloque 5 · Funciones Esenciales

Funciones que aparecen siempre

No es memorizar todas — es saber cuales resolver cada problema de negocio

CASE WHEN — el IF de SQL

```
SELECT nombre, segmento,
       CASE segmento
         WHEN 'VIP'          THEN 'Cliente oro'      -- nivel maximo
         WHEN 'Frecuente'    THEN 'Cliente plata'
         ELSE                 'Cliente bronce'      -- cualquier otro valor
       END AS categoria_cliente
FROM clientes;
```

Funciones de fecha mas usadas

```
SELECT fecha_pago,
       YEAR(fecha_pago) AS anio,
       MONTH(fecha_pago) AS mes,
       DATEDIFF(CURRENT_DATE, fecha_pago) AS dias
FROM pagos;
```

Funciones de texto mas usadas

```
SELECT
  UPPER(nombre)           AS nombre_mayus,
  LOWER(email)            AS email_minus,
  LENGTH(nombre)          AS longitud,
  TRIM(nombre)            AS sin_espacios,
  CONCAT(nombre, ' - ', ciudad) AS etiqueta
FROM clientes;
```

CASE WHEN es el condicional de SQL: crea categorias, aplica logica de negocio — todo dentro de un SELECT.

Repaso visual + pregunta de negocio + ingeniería inversa

Bloque 6 · Esquema Estrella

El Esquema Estrella — las tablas que ya tienes

fact_ventas en el centro conectada a 4 dimensiones via FOREIGN KEY



La tabla de hechos tiene todas las FK. Las dimensiones solo tienen PK y sus propios atributos.

PREGUNTA DE NEGOCIO

Pregunta #3 — Esquema Estrella

¿Que categoria de producto genero mas ingresos
en las tiendas de Medellin?

Muestra categoria, ingresos totales y unidades vendidas.

SOLUCION

Solucion — Pregunta #3

```
SELECT dp.categoria,  
       SUM(fv.total_venta) AS ingresos_totales,  
       SUM(fv.cantidad)   AS unidades_vendidas  
FROM fact_ventas fv  
JOIN dim_producto dp ON fv.producto_id = dp.producto_id  
JOIN dim_tienda   dt ON fv.tienda_id   = dt.tienda_id  
WHERE dt.ciudad = 'Medellin'  
GROUP BY dp.categoria  
ORDER BY ingresos_totales DESC;
```

Repaso completo — todo lo que ya sabes

SELECT / FROM / WHERE

La base de toda query. Orden de ejecucion: FROM → WHERE → SELECT. * = todas las columnas.

ORDER BY / DISTINCT / LIMIT

ORDER BY ASC/DESC. DISTINCT elimina duplicados. LIMIT acota filas. Alias con AS.

GROUP BY + Agregaciones

COUNT, SUM, AVG, MIN, MAX. HAVING filtra grupos. Regla: SELECT solo tiene columnas del GROUP BY o funciones.

JOINS

INNER (solo coincidencias) · LEFT (todos los de la izquierda) · RIGHT · FULL OUTER. ON define la condicion de union.

Subqueries y CTEs

Subquery en WHERE o FROM. CTE con WITH: mas legible, reutilizable. Elegir segun complejidad.

Funciones

CASE WHEN (condicional) · YEAR/MONTH/DATEDIFF (fechas) · UPPER/CONCAT (texto)

Esquema Estrella

fact_ventas en el centro con FK a 4 dims. Ingenieria inversa: leer las FOREIGN KEY del DDL = armar el diagrama.

Si entiendes estos 7 bloques, estas listo. La evaluacion mide logica, no memorizar funciones.